1. **Creating the database :**

- **Sql query:**

Create database ecommerce_analytics ;

2. **Creating the tables :**

1) Customers Table:

Query | Query History

```
1  CREATE TABLE customers (
2      customer_id TEXT PRIMARY KEY,
3      customer_unique_id TEXT,
4      customer_zip_code_prefix INTEGER,
5      customer_city TEXT,
6      customer_state TEXT
7  );
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 120 msec.
```

2) Orders Table:

Query | Query History

```sql
CREATE TABLE orders (
    order_id TEXT PRIMARY KEY,

    customer_id TEXT,

    order_status TEXT,

    order_purchase_timestamp TIMESTAMP,

    order_approved_at TIMESTAMP,

    order_delivered_carrier_date TIMESTAMP,

    order_delivered_customer_date TIMESTAMP,

    order_estimated_delivery_date TIMESTAMP,

    CONSTRAINT fk_customer
        FOREIGN KEY(customer_id)
        REFERENCES customers(customer_id)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 111 msec.
```

3) Products table:

Query | Query History

```sql
CREATE TABLE products (
    product_id TEXT PRIMARY KEY,

    product_category_name TEXT,

    product_name_lenght INTEGER,

    product_description_lenght INTEGER,

    product_photos_qty INTEGER,

    product_weight_g NUMERIC,

    product_length_cm NUMERIC,

    product_height_cm NUMERIC,

    product_width_cm NUMERIC
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 114 msec.
```

---

4) <u>Sellers table :</u>

Query | Query History

```sql
CREATE TABLE sellers (
    seller_id TEXT PRIMARY KEY,

    seller_zip_code_prefix INTEGER,

    seller_city TEXT,

    seller_state TEXT
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 102 msec.
```

---

5) <u>Order Items Table :</u>

Query | Query History

```sql
CREATE TABLE order_items (

    order_id TEXT,

    order_item_id INTEGER,

    product_id TEXT,

    seller_id TEXT,

    shipping_limit_date TIMESTAMP,

    price NUMERIC(10,2),

    freight_value NUMERIC(10,2),

    PRIMARY KEY(order_id, order_item_id),

    FOREIGN KEY(order_id)
        REFERENCES orders(order_id),

    FOREIGN KEY(product_id)
        REFERENCES products(product_id),

    FOREIGN KEY(seller_id)
        REFERENCES sellers(seller_id)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 109 msec.
```

6) Payments table:

Query | Query History

```sql
CREATE TABLE payments (

    order_id TEXT,

    payment_sequential INTEGER,

    payment_type TEXT,

    payment_installments INTEGER,

    payment_value NUMERIC(10,2),

    PRIMARY KEY(order_id, payment_sequential),

    FOREIGN KEY(order_id)
        REFERENCES orders(order_id)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 108 msec.
```

---

7) Category_transalation table:

Query | Query History

```sql
CREATE TABLE category_translation (

    product_category_name TEXT PRIMARY KEY,

    product_category_name_english TEXT

);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 90 msec.
```

---

8) Reviews Table :

Query | Query History

```
CREATE TABLE reviews (

    review_id TEXT,

    order_id TEXT,

    review_score INTEGER,

    review_comment_title TEXT,

    review_comment_message TEXT,

    review_creation_date TIMESTAMP,

    review_answer_timestamp TIMESTAMP,

    PRIMARY KEY (review_id, order_id),

    FOREIGN KEY (order_id)
        REFERENCES orders(order_id)

);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 169 msec.
```

---

9) Geolocation table:

Query | Query History

```
1   CREATE TABLE geolocation (
2
3       geolocation_zip_code_prefix INTEGER,
4
5       geolocation_lat NUMERIC,
6
7       geolocation_lng NUMERIC,
8
9       geolocation_city TEXT,
10
11      geolocation_state TEXT
12
13  );
```

Data Output | Messages | Notifications

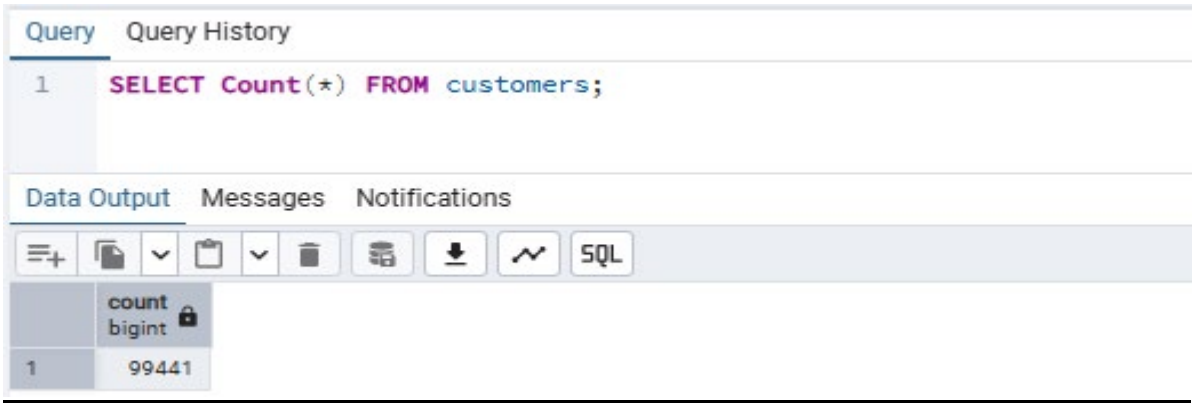
```
CREATE TABLE

Query returned successfully in 107 msec.
```

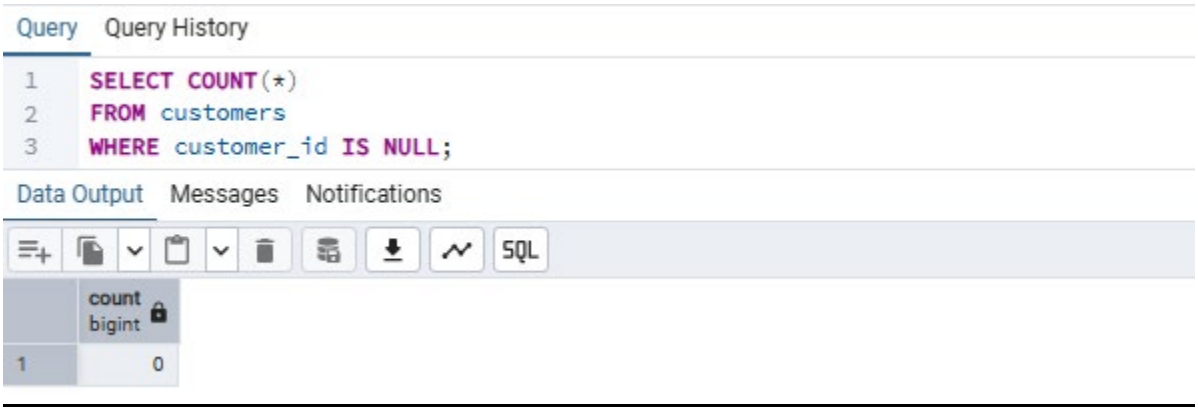
3. Importing the data into the tables:

We imported the data using pgAdmin UI , and all tables loaded successfully .

4. Data Validation :

1) Customers table:





- customers Validation:

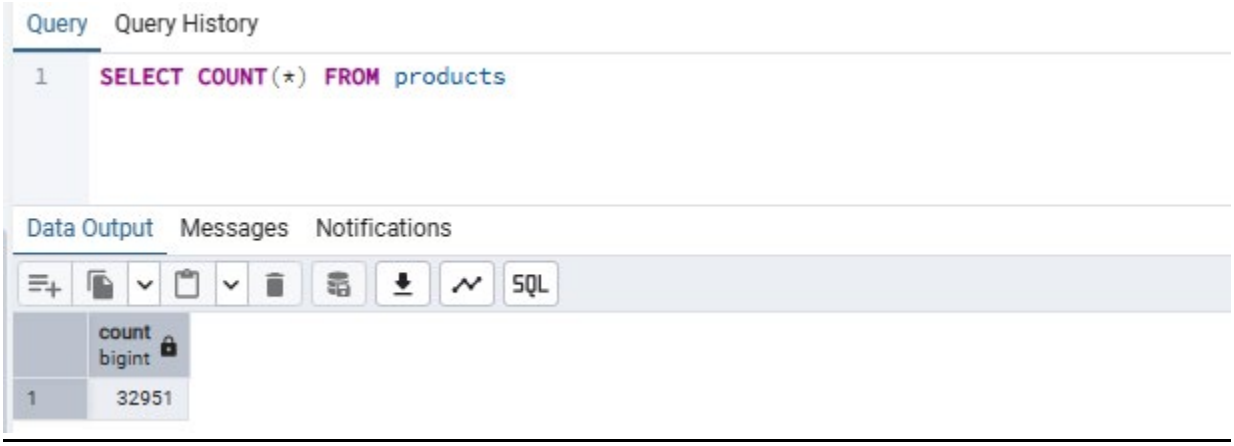
- Row count verified

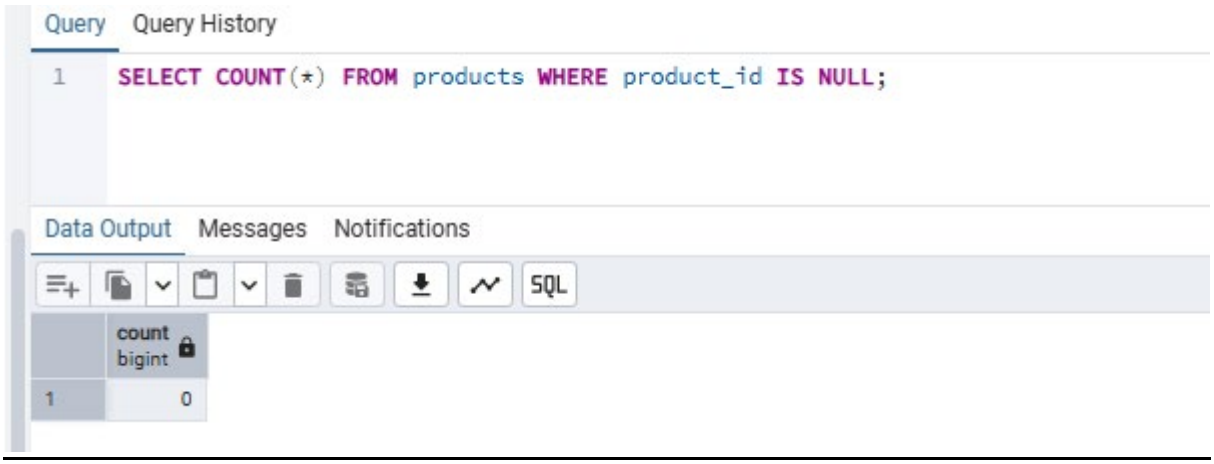
- No null customer_id values

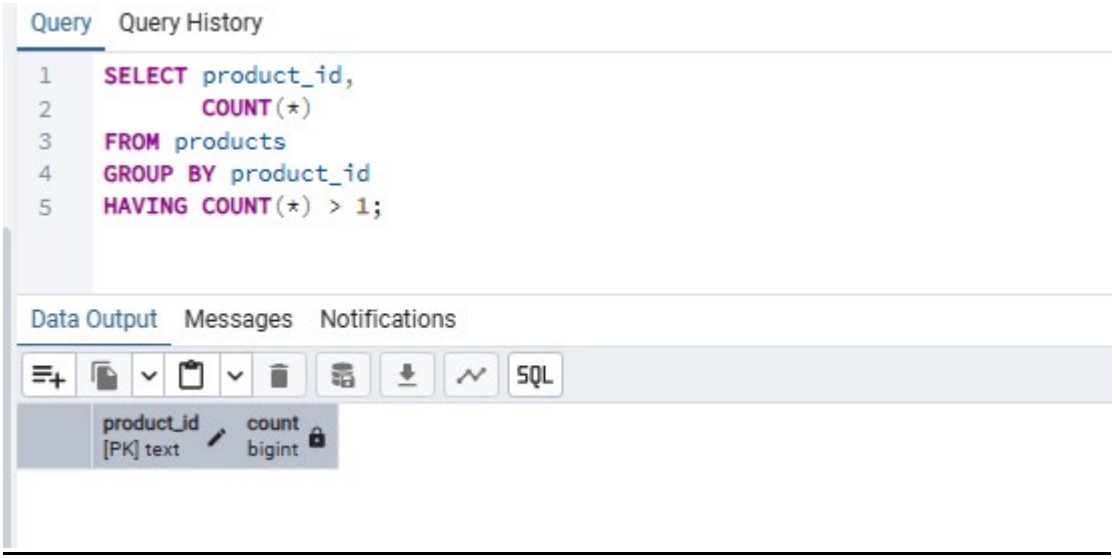
- No duplicate primary keys

customers table passed validation checks.

2) Product table:







- products Validation:

- Row count verified

- No null product_id values

- No duplicate primary keys

products table passed validation checks.

3) Sellers table:

Query Query History

```
1  SELECT COUNT(*) FROM sellers;
```

Data Output Messages Notifications

SQL

	count bigint
1	3095

Query Query History

```
1  SELECT COUNT(*) FROM sellers WHERE seller_id IS NULL;
```

Data Output Messages Notifications

SQL

	count bigint
1	0

Query Query History

```
1  SELECT seller_id,
2         COUNT(*)
3  FROM sellers
4  GROUP BY seller_id
5  HAVING COUNT(*) > 1;
```

Data Output Messages Notifications

SQL

	seller_id [PK] text	count bigint

- sellers Validation:

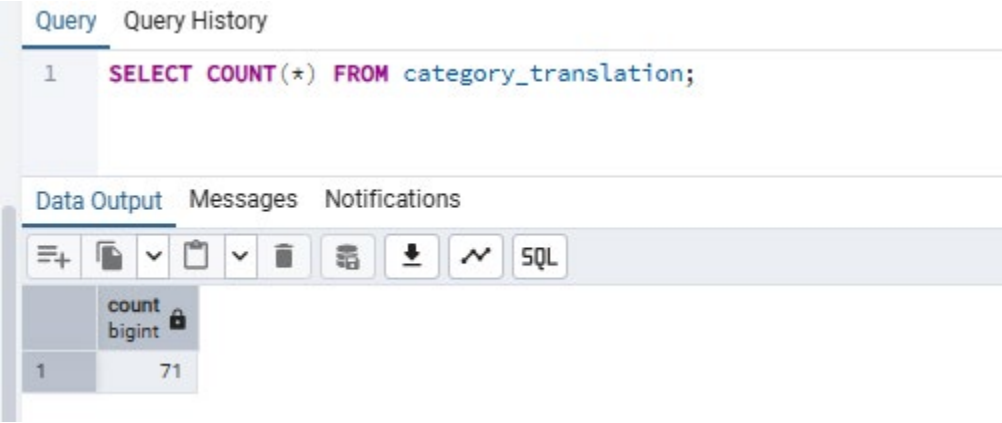
- Row count verified

- No null seller_id values

- No duplicate primary keys

sellers table passed validation checks.

4) Category_translation table:

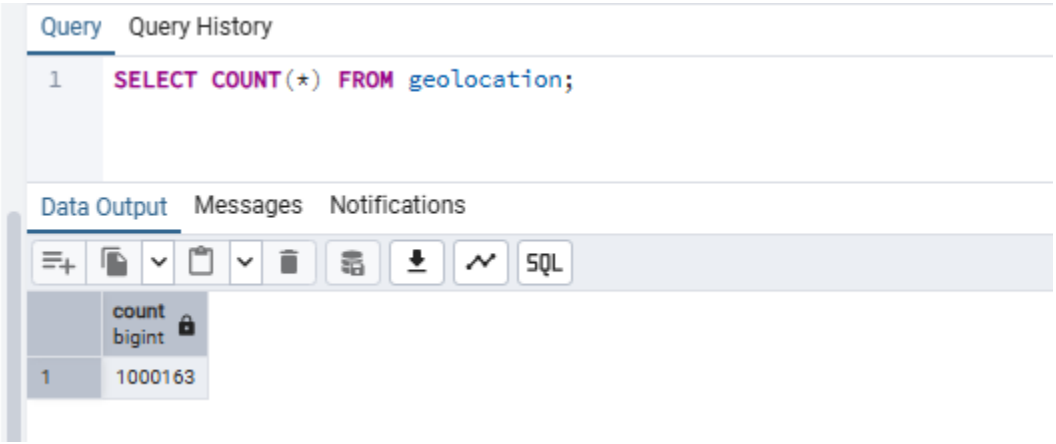


- category_translation Validation:

- Row count verified

Category_translation table passed validation checks.

5) Geolocation table:

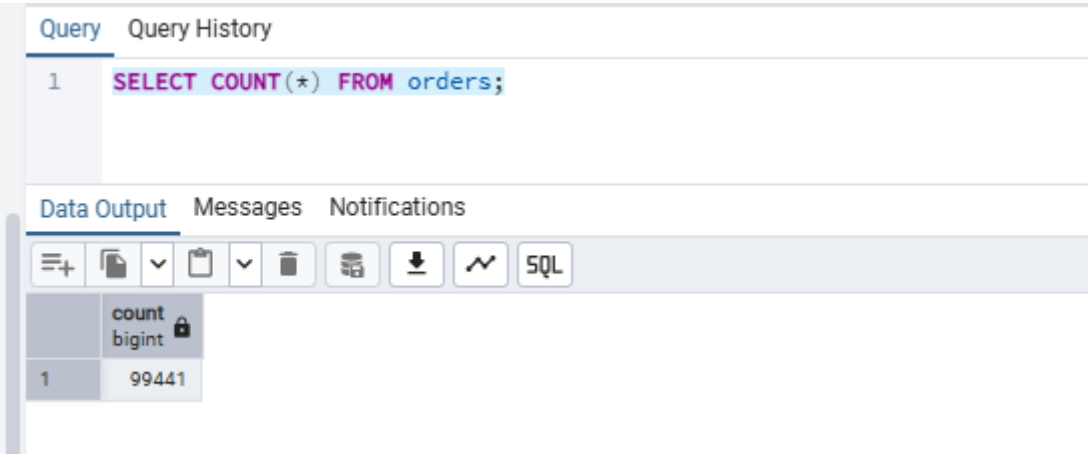


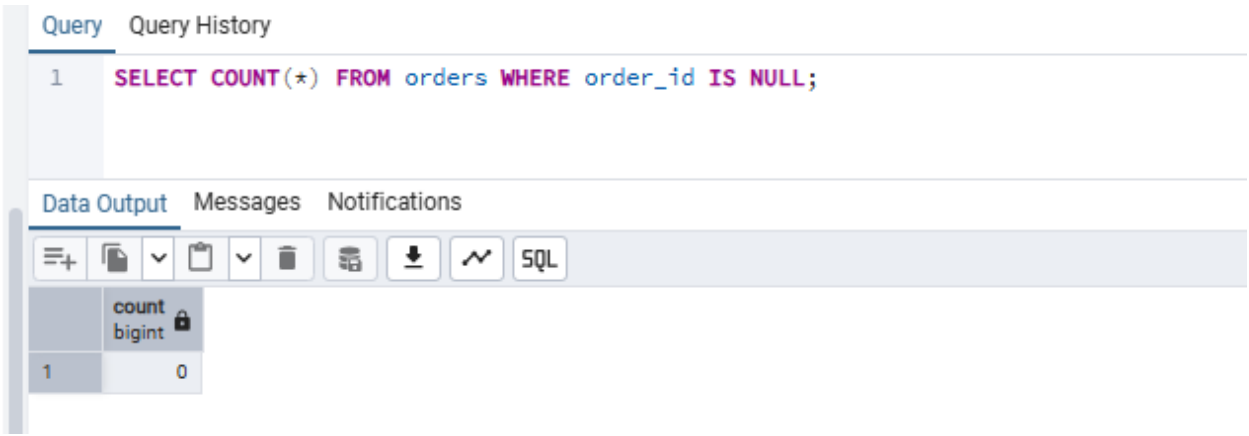
- geoloction Validation:

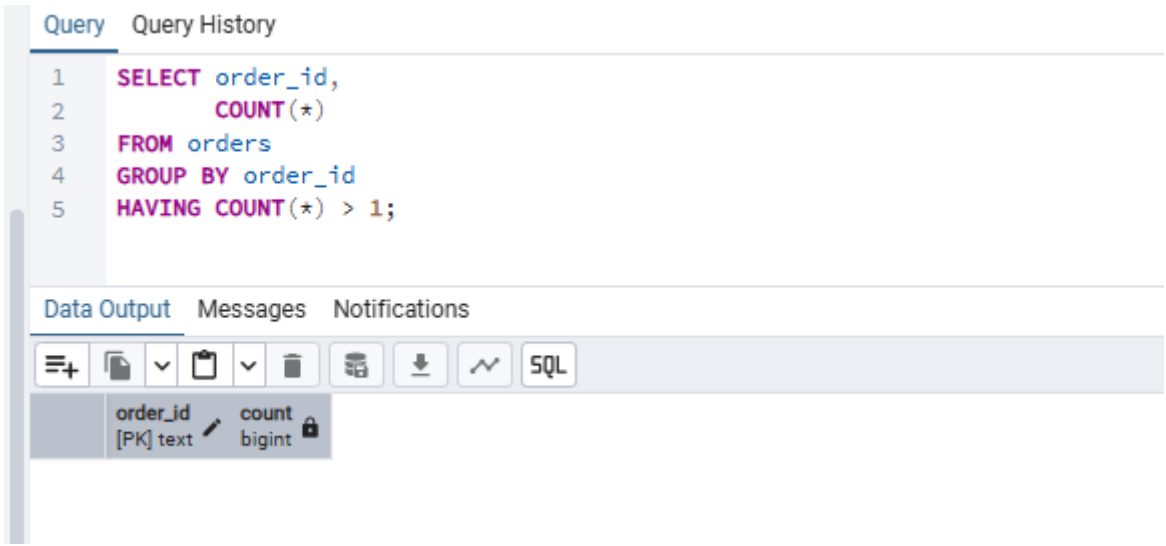
- Row count verified

geolocation table passed validation checks.

6) Orders table :







- orders Validation:

- Row count verified

- No null order_id values

- No duplicate primary keys

orders table passed validation checks.

7) Payments table :

Query | Query History

```
1  SELECT COUNT(*) FROM payments;
```

Data Output | Messages | Notifications

	count bigint
1	103886

Query | Query History

```
1  SELECT COUNT(*)
2  FROM payments
3  WHERE order_id IS NULL;
```

Data Output | Messages | Notifications

	count bigint
1	0

Query | Query History

```
1  SELECT
2      order_id,
3      payment_sequential,
4      COUNT(*)
5  FROM payments
6  GROUP BY order_id, payment_sequential
7  HAVING COUNT(*) > 1;
```

Data Output | Messages | Notifications

	order_id [PK] text	payment_sequential [PK] integer	count bigint

- Payment Validation:

- Row count verified

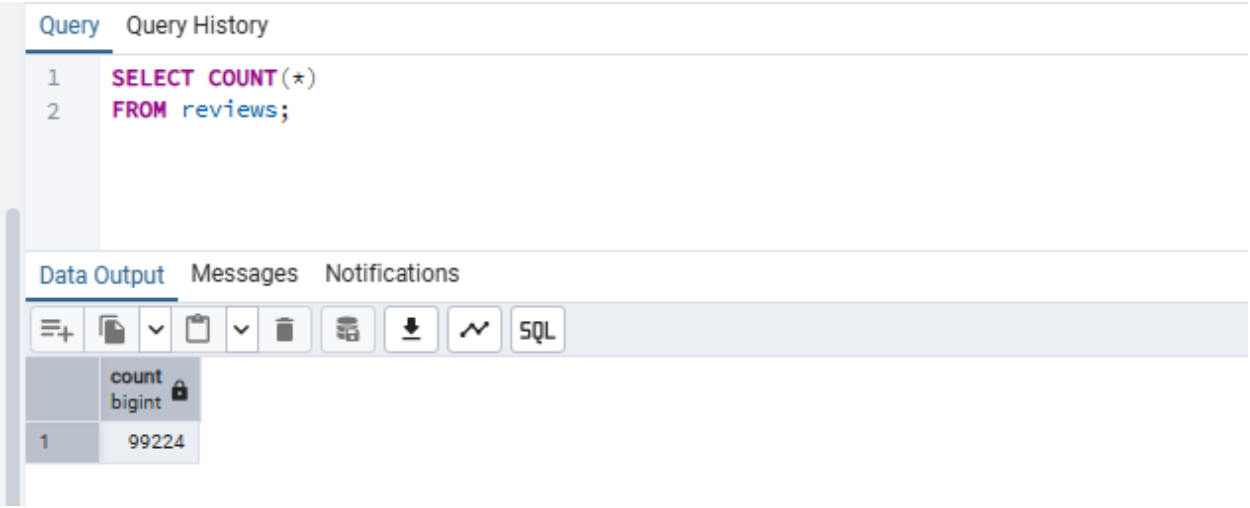
- No null order_id values

- No duplicate composite primary keys

- No negative payment values.

payments table passed validation checks.

8) Reviews table :



Query Query History

```sql
SELECT COUNT(*)
FROM reviews
WHERE order_id IS NULL;
```

Data Output Messages Notifications

	count bigint
1	0

Query Query History

```sql
SELECT COUNT(*)
FROM reviews
WHERE review_score IS NULL;
```

Data Output Messages Notifications

	count bigint
1	0

Query Query History

```sql
SELECT
    review_id,
    order_id,
    COUNT(*)
FROM reviews
GROUP BY review_id, order_id
HAVING COUNT(*) > 1;
```

Data Output Messages Notifications

	review_id [PK] text	order_id [PK] text	count bigint

- Reviews Validation:

- Row count verified

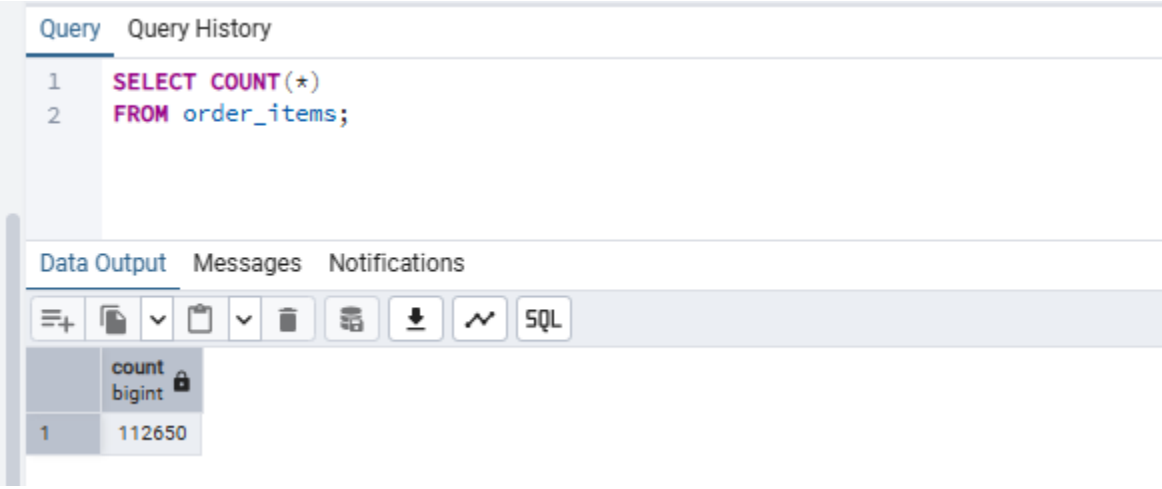
- No null order_id values

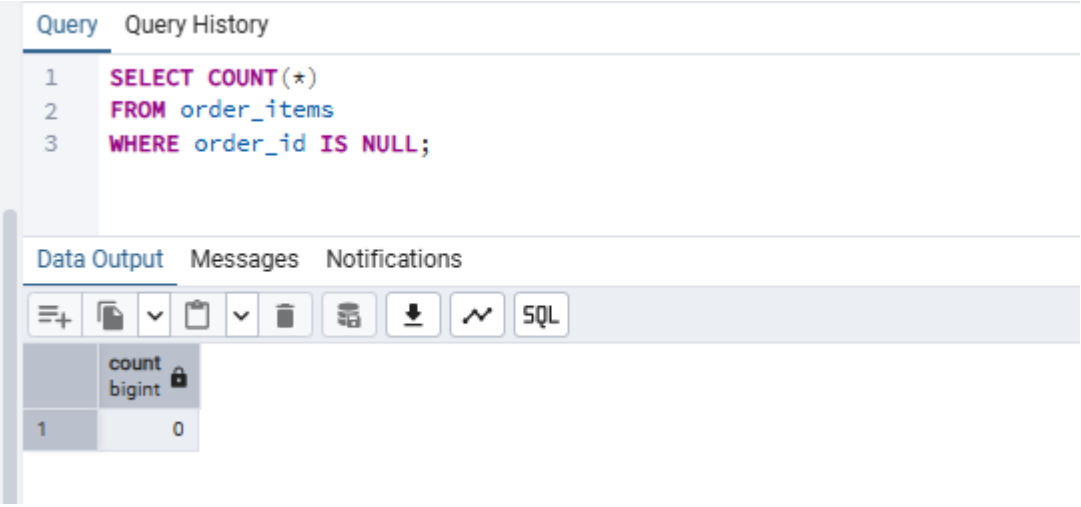
- No duplicate primary keys

- Review scores successfully loaded

Reviews table passed validation checks.

9) Order_items table :





Query | Query History

```
1  SELECT COUNT(*)
2  FROM order_items
3  WHERE product_id IS NULL;
```

Data Output | Messages | Notifications

SQL

	count bigint
1	0

Query | Query History

```
1  SELECT
2      order_id,
3      order_item_id,
4      COUNT(*)
5  FROM order_items
6  GROUP BY order_id, order_item_id
7  HAVING COUNT(*) > 1;
```

Data Output | Messages | Notifications

SQL

	order_id [PK] text	order_item_id [PK] integer	count bigint

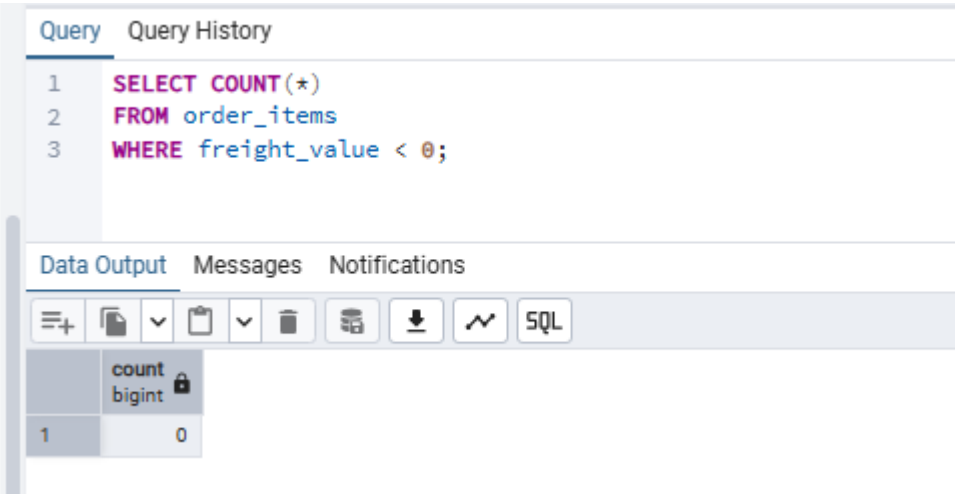
Query | Query History

```
1  SELECT COUNT(*)
2  FROM order_items
3  WHERE price < 0;
```

Data Output | Messages | Notifications

SQL

	count bigint
1	0



- Order_items_Validation:

- Row count verified

- No null order_id values

-No null product_id values

- No duplicate primary keys

- No negative price values.

- No negative freight value.

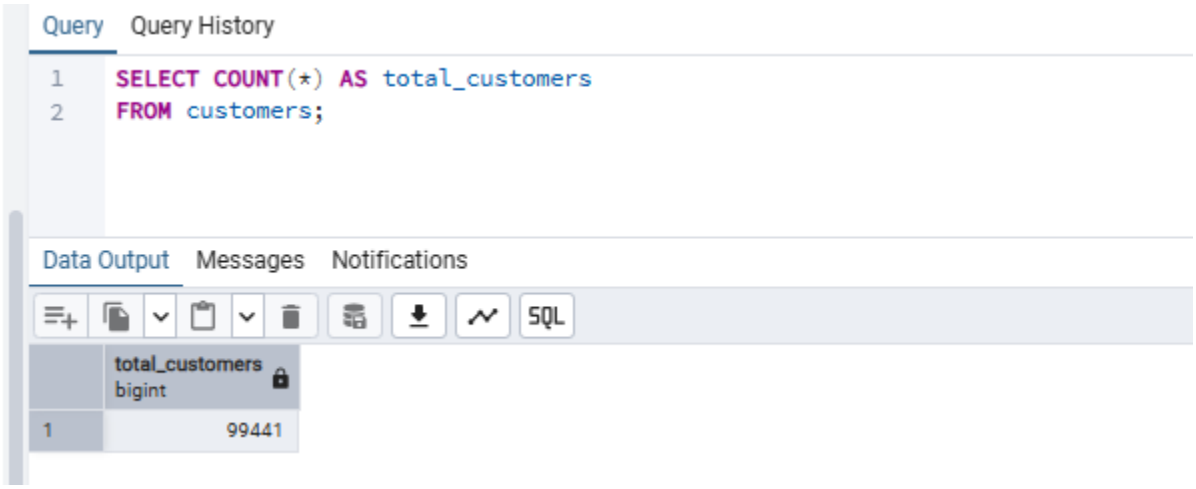
Order_items table passed validation checks.

5. Exploratory SQL Analysis (EDA) :

5.1 : Data base overview (Understanding the business) :

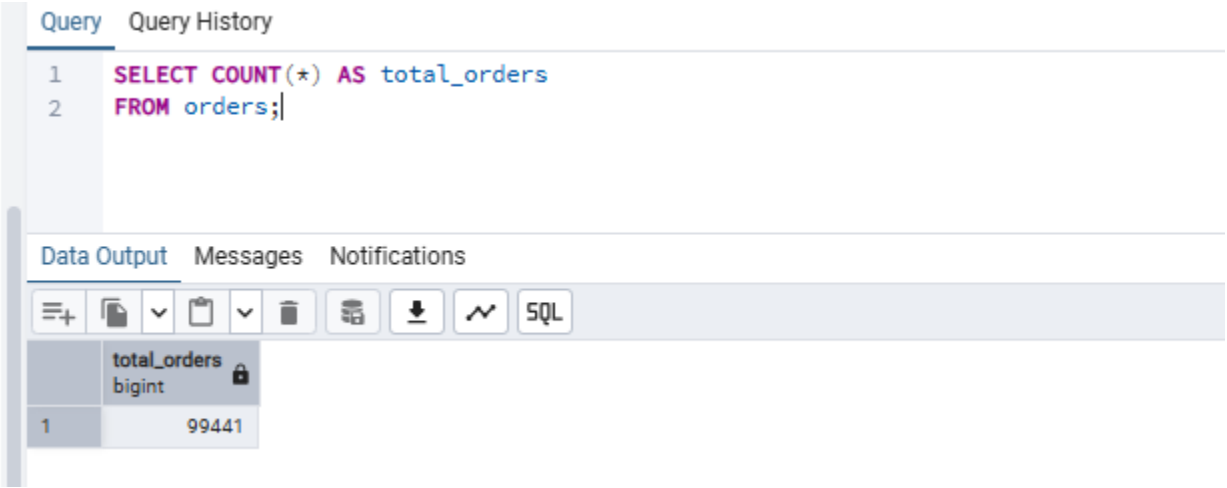
- How big is this business?

Question 1 : How many customers do we have?

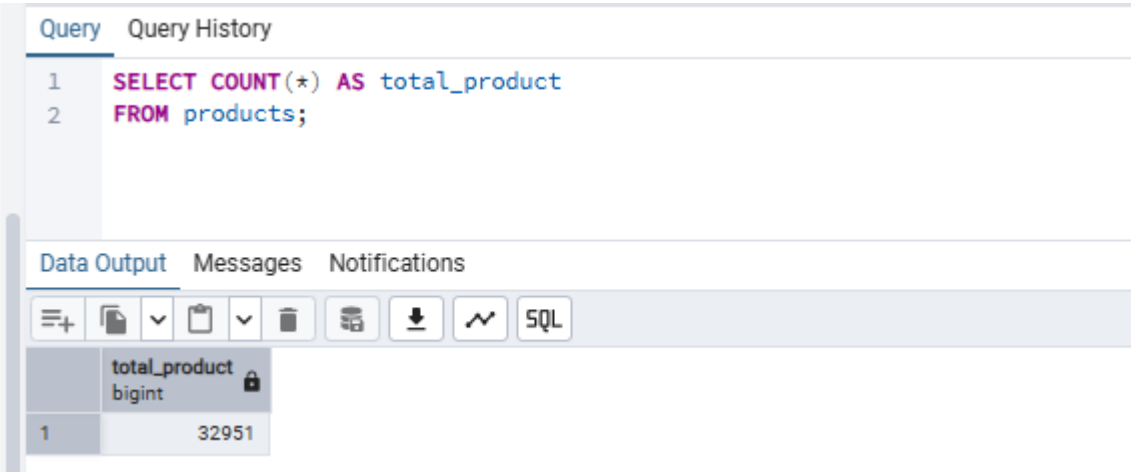


- The company served nearly 100k customers,
- indicating a large-scale e-commerce operation.

Question 2 : How many orders ?

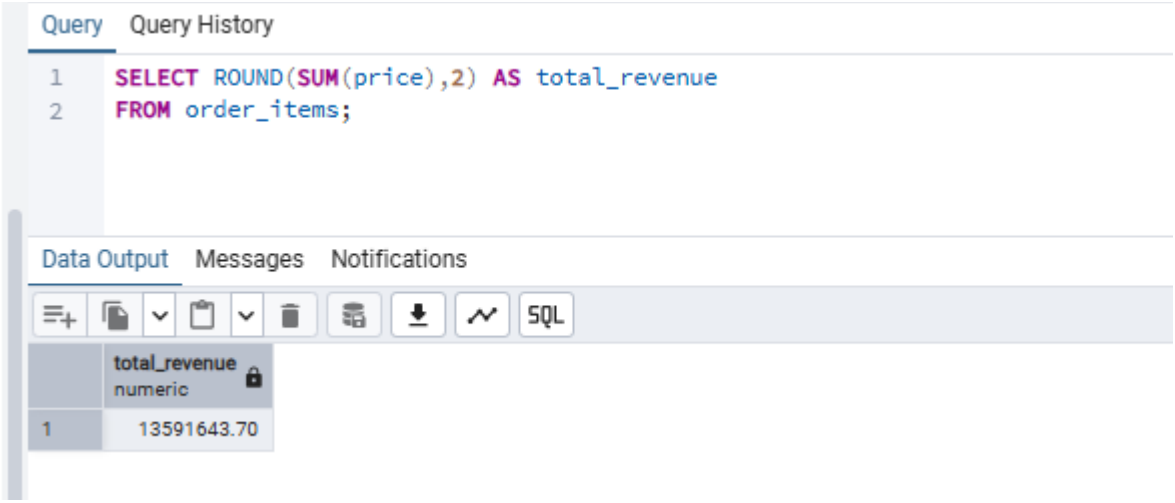


Question 3 : How many product we have ?



Question 5 : How many sellers ?

Question 6 : what is the total revenue generated ?



Finding

The platform generated substantial revenue across the analyzed period.

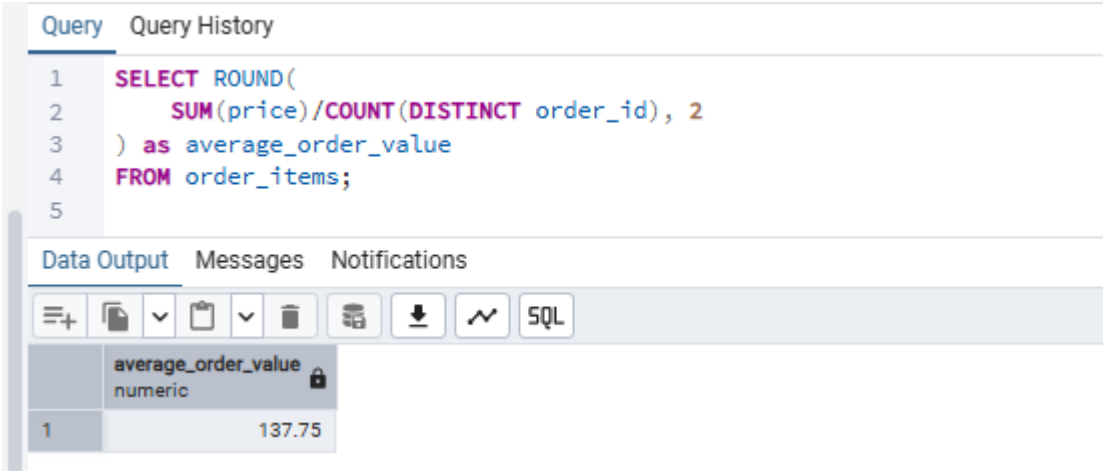
Business Insight

Revenue performance indicates a healthy transaction volume and demonstrates the platform's ability to attract and convert customers at scale.

Question 7 : what is the average order value ?

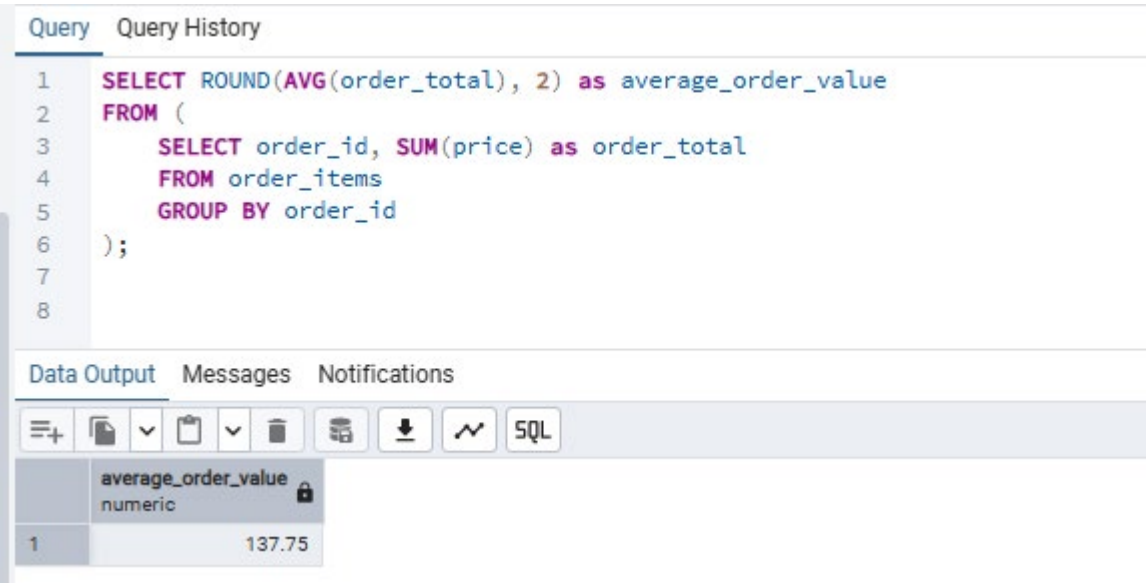
Method 1:

AOV = Total Revenue / Total Orders



Method 2:

Calculate each order total, then average all order totals.



- Both methods produced the same result, confirming the KPI calculate-on.
- AOV indicates how much customers spend per purchase on average.

The average order value provides an estimate of customer spending per purchase.

Business Insight

Monitoring AOV is critical because increasing customer basket size can often generate revenue growth more efficiently than acquiring new customers.

Question 7 : What is the average items per order ?

```
SELECT ROUND(AVG(items_per_order), 0) as average_item_per_order
FROM (
    SELECT order_id, COUNT(order_item_id) as items_per_order
    FROM order_items
    GROUP BY order_id
);
```

	average_item_per_order numeric
1	1

Method 2:

```
SELECT ROUND(
    COUNT(order_item_id) / COUNT(DISTINCT order_id), 0) as average_item_per_order
FROM order_items;
```

	average_item_per_order numeric
1	1

- We can tell that most of the orders contain 1 item per order.

Question 8 : getting the orders by status ?

Query | Query History

```
1  SELECT order_status,
2      COUNT(order_id) as orders_by_status
3  FROM orders
4  GROUP BY order_status
5  ORDER BY orders_by_status DESC;
6
```

Data Output | Messages | Notifications

	order_status text	orders_by_status bigint
1	delivered	96478
2	shipped	1107
3	canceled	625
4	unavailable	609
5	invoiced	314
6	processing	301
7	created	5
8	approved	2

- Most orders are successfully delivered, indicating an efficient fulfillment process.
- But also seeing both canceled and unavailable a bit high too that may indicate inventory or operational issues.

Question 9 : what is revenue including the shipping cost ?

Query Query History

```
1  SELECT ROUND(
2      SUM(price + freight_value), 2
3  ) as total_business_revenue
4  FROM order_items;
5
```

Data Output Messages Notifications

SQL

	total_business_revenue numeric
1	15843553.24

Question 10 : What is the shipping percentage contribution to total revenue?

Query Query History

```
1  SELECT ROUND(
2      SUM(freight_value) * 100 /
3      SUM(price + freight_value), 2
4  ) as shipping_percentage
5  FROM order_items;
6
```

Data Output Messages Notifications

SQL

	shipping_percentage numeric
1	14.21

6. Customer Analysis :

6.1- Customer geography:

Question 11 : Top 10 Customer states :

Query | Query History

```sql
SELECT customer_state, COUNT(customer_id) as total_customer_by_state
FROM customers
GROUP BY customer_state
order BY total_customer_by_state DESC
LIMIT 10;
```

Data Output | Messages | Notifications

	customer_state text	total_customer_by_state bigint
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

- Sao Paulo dominates customer acquisition, maybe marketing investments should continue there.

Question 12 : Top 10 Customer Cities ?

Query | Query History

```sql
SELECT customer_city, COUNT(customer_id) as total_customer_by_city
FROM customers
GROUP BY customer_city
order BY total_customer_by_city DESC
LIMIT 10;
```

Data Output | Messages | Notifications

	customer_city text	total_customer_by_city bigint
1	sao paulo	15540
2	rio de janeiro	6882
3	belo horizonte	2773
4	brasilia	2131
5	curitiba	1521
6	campinas	1444
7	porto alegre	1379
8	salvador	1245
9	guarulhos	1189
10	sao bernardo do cam...	938

6.2 - Customer Order Behavior :

Question 13 : Customers who placed orders ?

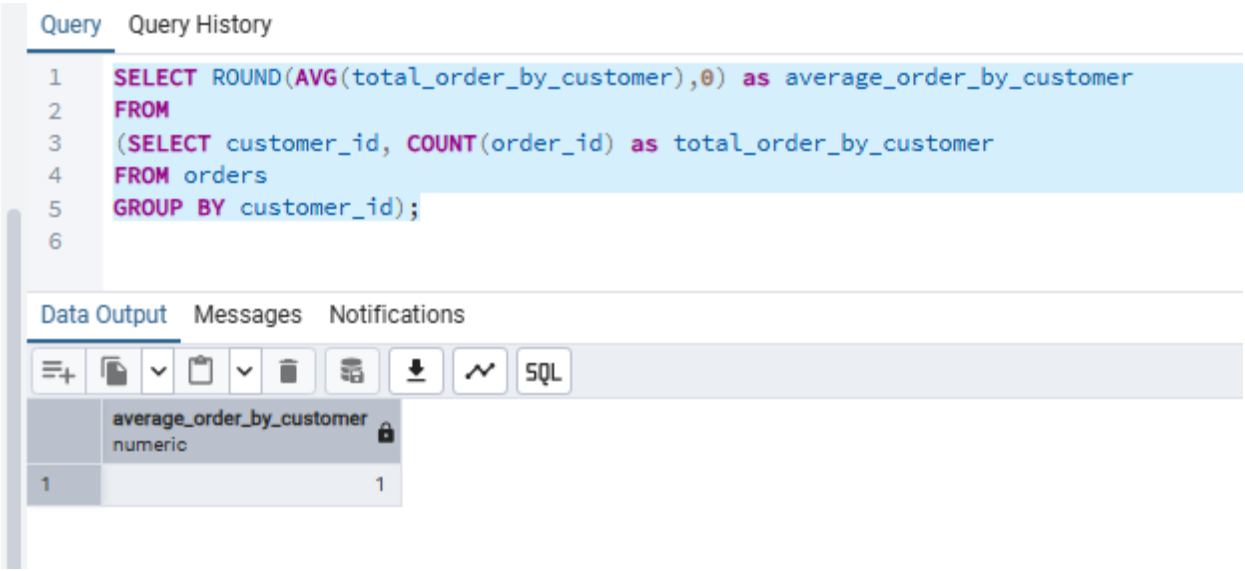
Query | Query History

```sql
SELECT COUNT(DISTINCT c.customer_id)
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

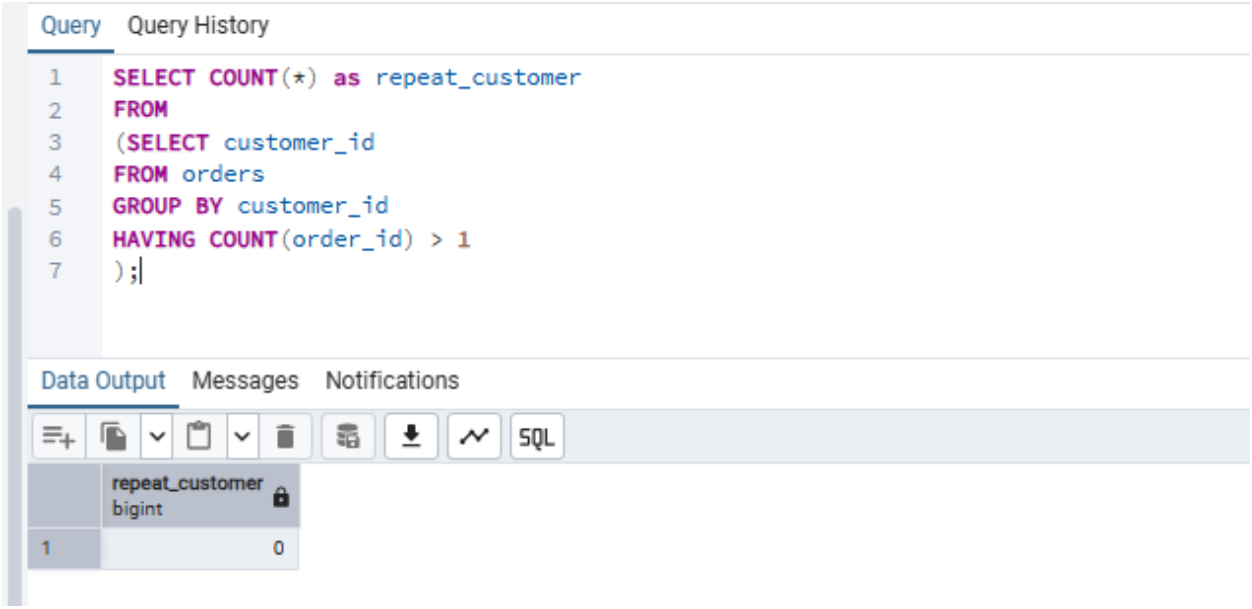
Data Output | Messages | Notifications

	count bigint
1	99441

Question 14 : Average Orders By Customer ?



Question 15 : is there repeat customers ?



- We can tell that there is no repeated customers that do more than order.
- Business relies heavily on acquiring new customers.

6.3 – Customer revenue analysis:

Question 16 : Top Customers by revenue ?

Query | Query History

```
1  SELECT o.customer_id, ROUND(SUM(price),2) AS revenue_by_customer
2  FROM orders o
3  INNER JOIN order_items ot
4  ON o.order_id = ot.order_id
5  GROUP BY customer_id
6  ORDER BY revenue_by_customer DESC
7  LIMIT 10;
```

Data Output | Messages | Notifications

SQL

	customer_id text	revenue_by_customer numeric
1	1617b1357756262bfa56ab541c47bc...	13440.00
2	ec5b2ba62e574342386871631fafd3fc	7160.00
3	c6e2731c5b391845f6800c97401a43...	6735.00
4	f48d464a0baaea338cb25f816991ab1f	6729.00
5	3fd6777bbce08a352fddd04e4a7cc8f6	6499.00
6	05455dfa7cd02f13d132aa7a6a9729...	5934.60
7	df55c14d1476a9a3467f131269c2477f	4799.00
8	24bbf5fd2f2e1b359ee7de94defc4a15	4690.00
9	e0a2412720e9ea4f26c1ac985f6a73...	4599.90
10	3d979689f636322c62418b6346b1c6...	4590.00

Question 17: Average revenue by customer ?

Query Query History

```
1  SELECT ROUND(AVG(revenue_by_customer),2) as average_revenue_by_customer
2  FROM
3  (SELECT o.customer_id, ROUND(SUM(price),2) AS revenue_by_customer
4  FROM orders o
5  INNER JOIN order_items ot
6  ON o.order_id = ot.order_id
7  GROUP BY customer_id);
8
```

Data Output Messages Notifications

SQL

	average_revenue_by_customer numeric
1	137.75

6.4 – Customer Satisfaction:

Question 18: Average Review Score?

Query Query History

```
1  SELECT ROUND(AVG(review_score),2) as average_review_score
2  FROM reviews;
```

Data Output Messages Notifications

SQL

	average_review_score numeric
1	4.09

Question 19: Review distribution?

Query | Query History

```
1  SELECT review_score, COUNT(review_id) as total_reviews
2  FROM reviews
3  GROUP BY review_score
4  ORDER BY review_score;
```

Data Output | Messages | Notifications

	review_score integer	total_reviews bigint
1	1	11424
2	2	3151
3	3	8179
4	4	19142
5	5	57328

Question 20: Average review by state?

Query | Query History

```sql
SELECT c.customer_state, ROUND(AVG(r.review_score),2) as average_review_by_state
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN reviews r ON o.order_id = r.order_id
GROUP BY c.customer_state;
```

Data Output | Messages | Notifications

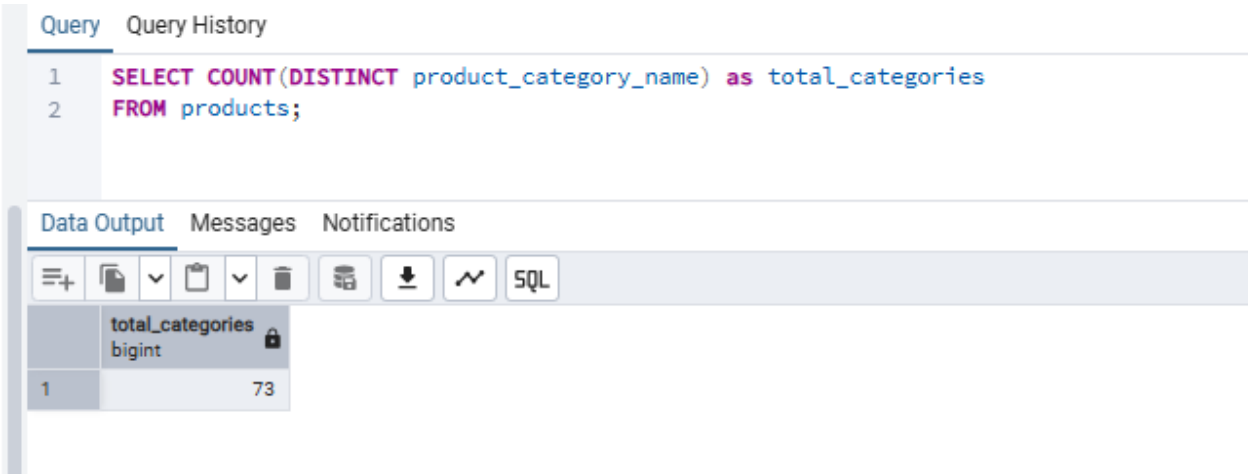
	customer_state text	average_review_by_state numeric
1	AC	4.05
2	AL	3.75
3	AM	4.18
4	AP	4.19
5	BA	3.86
6	CE	3.85
7	DF	4.06
8	ES	4.04
9	GO	4.04
10	MA	3.76
11	MG	4.14
12	MS	4.12
13	MT	4.10
14	PA	3.85
15	PB	4.02
16	PE	4.01

- Some of theme are happier and some they are least satisfied this :

Could indicate: delivery issues , logistics problems , seller performance differences

7. Product Analysis :

Question 21: Total number of product categories?



- More categories like this : means More product diversity and Broader customer reach.

Question 22: Top 10 best-selling categories?

Query | Query History

```sql
SELECT p.product_category_name, COUNT(oi.order_id) as total_orders_by_category
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY total_orders_by_category DESC
LIMIT 10;
```

Data Output | Messages | Notifications

	product_category_name text	total_orders_by_category bigint
1	cama_mesa_banho	11115
2	beleza_saude	9670
3	esporte_lazer	8641
4	moveis_decoracao	8334
5	informatica_acessorios	7827
6	utilidades_domesticas	6964
7	relogios_presentes	5991
8	telefonia	4545
9	ferramentas_jardim	4347
10	automotivo	4235

- We can tell that cama_mesa_banho category its the Highest demand category

Question 22: Top revenue categories?

Query Query History

```sql
SELECT p.product_category_name, ROUND(SUM(oi.price),2) as revenue_by_category
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY revenue_by_category DESC
LIMIT 10;
```

Data Output Messages Notifications

SQL

	product_category_name text	revenue_by_category numeric
1	beleza_saude	1258681.34
2	relogios_presentes	1205005.68
3	cama_mesa_banho	1036988.68
4	esporte_lazer	988048.97
5	informatica_acessorios	911954.32
6	moveis_decoracao	729762.49
7	cool_stuff	635290.85
8	utilidades_domesticas	632248.66
9	automotivo	592720.11
10	ferramentas_jardim	485256.46

Question 23 : Average Product Price By Category?

Query Query History

```sql
SELECT p.product_category_name, ROUND(AVG(oi.price),2) as avg_price_by_category
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY avg_price_by_category DESC;
```

Data Output Messages Notifications

SQL

	product_category_name text	avg_price_by_category numeric
1	pcs	1098.34
2	portateis_casa_forno_e_cafe	624.29
3	eletrodomesticos_2	476.12
4	agro_industria_e_comercio	342.12
5	instrumentos_musicais	281.62
6	eletroportateis	280.78
7	portateis_cozinha_e_preparadores_de_aliment...	264.57
8	telefonia_fixa	225.69
9	construcao_ferramentas_seguranca	208.99
10	relogios_presentes	201.14
11	climatizacao	185.27
12	moveis_quarto	183.75
13	pc_gamer	171.77
14	cool_stuff	167.36

- We can see that pc’s have the highst prices , that’s logical

Same but with category translated in English:

Query | Query History

```sql
SELECT c.product_category_name_english, ROUND(AVG(oi.price),2) as avg_price_by_category
FROM category_translation c
INNER JOIN products p ON c.product_category_name = p.product_category_name
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY c.product_category_name_english
ORDER BY avg_price_by_category DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 71

	product_category_name_english text	avg_price_by_category numeric
1	computers	1098.34
2	small_appliances_home_oven_and_c...	624.29
3	home_appliances_2	476.12
4	agro_industry_and_commerce	342.12
5	musical_instruments	281.62
6	small_appliances	280.78
7	fixed_telephony	225.69
8	construction_tools_safety	208.99
9	watches_gifts	201.14
10	air_conditioning	185.27
11	furniture_bedroom	183.75
12	cool_stuff	167.36
13	kitchen_dining_laundry_garden_furnit...	164.87
14	office_furniture	162.01

Question 24: Top 10 Individual product by revenue?

Query | Query History

```sql
SELECT product_id, ROUND(SUM(price),2) as revenue_by_product
FROM order_items
GROUP BY product_id
ORDER BY revenue_by_product DESC
LIMIT 10;
```

Data Output | Messages | Notifications

	product_id text	revenue_by_product numeric
1	bb50f2e236e5eea01006801376546...	63885.00
2	6cdd53843498f92890544667809f15...	54730.20
3	d6160fb7873f184099d9bc95e30376...	48899.34
4	d1c427060a0f73f6b889a5c7c61f2a...	47214.51
5	99a4788cb24856965c36a24e339b6...	43025.56
6	3dd2a17168ec895c781a9191c1e95...	41082.60
7	25c38557cf793876c5abdd5931f922...	38907.32
8	5f504b3a1c75b73d6151be81eb05b...	37733.90
9	53b36df67ebb7c41585e8d54d6772...	37683.42
10	aca2eb7d00ea1a7b8ebd4e6831466...	37608.90

Question 25: top product by quantity sold ?

Query | Query History

```sql
SELECT p.product_id, COUNT(oi.order_id) as quantity_sold_by_product
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id
ORDER BY quantity_sold_by_product DESC
LIMIT 10;
```

Data Output | Messages | Notifications

	product_id [PK] text	quantity_sold_by_product bigint
1	aca2eb7d00ea1a7b8ebd4e68314663...	527
2	99a4788cb24856965c36a24e339b6...	488
3	422879e10f46682990de24d770e7f8...	484
4	389d119b48cf3043d311335e499d9c...	392
5	368c6c730842d78016ad823897a37...	388
6	53759a2ecddad2bb87a079a1f1519f...	373
7	d1c427060a0f73f6b889a5c7c61f2ac4	343
8	53b36df67ebb7c41585e8d54d6772e...	323
9	154e7e31ebfa092203795c972e5804...	281
10	3dd2a17168ec895c781a9191c1e95a...	274

Question 26: Product category revenue contribution rate % ?

Query | Query History

```sql
SELECT p.product_category_name, ROUND(SUM(oi.price) * 100 / (SELECT SUM(price) FROM order_items), 2)
as category_revenue_rate
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY category_revenue_rate DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 74

	product_category_name text	category_revenue_rate numeric
1	beleza_saude	9.26
2	relogios_presentes	8.87
3	cama_mesa_banho	7.63
4	esporte_lazer	7.27
5	informatica_acessorios	6.71
6	moveis_decoracao	5.37
7	cool_stuff	4.67
8	utilidades_domesticas	4.65
9	automotivo	4.36
10	ferramentas_jardim	3.57
11	brinquedos	3.56
12	bebes	3.03
13	perfumaria	2.94
14	telefonia	2.38

Total rows: 74 | Query complete 00:00:00.280

Question 27: Categories with highest review scores ?

Query | Query History

```sql
SELECT p.product_category_name, ROUND(AVG(r.review_score), 2) as avg_category_review_score
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
INNER JOIN reviews r ON oi.order_id = r.order_id
GROUP BY p.product_category_name
ORDER BY avg_category_review_score DESC
LIMIT 10;
```

Data Output | Messages | Notifications

Showing rows: 1 t

	product_category_name text	avg_category_review_score numeric
1	cds_dvds_musicais	4.64
2	fashion_roupa_infanto_juvenil	4.50
3	livros_interesse_geral	4.45
4	construcao_ferramentas_ferramen...	4.44
5	flores	4.42
6	livros_importados	4.40
7	livros_tecnicos	4.37
8	alimentos_bebidas	4.32
9	malas_acessorios	4.32
10	portateis_casa_forno_e_cafe	4.30

- We see that these categories delight customers.

Question 28 : Categories with lowest review score ?

The same query, we just change the order by direction :

Query | Query History

```sql
SELECT p.product_category_name, ROUND(AVG(r.review_score), 2) as avg_category_rev
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
INNER JOIN reviews r ON oi.order_id = r.order_id
GROUP BY p.product_category_name
ORDER BY avg_category_review_score ASC
LIMIT 10;
```

Data Output | Messages | Notifications

	product_category_name text	avg_category_review_score numeric
1	seguros_e_servicos	2.50
2	fraldas_higiene	3.26
3	portateis_cozinha_e_preparadores_de_alimen...	3.27
4	pc_gamer	3.33
5	moveis_escritorio	3.49
6	casa_conforto_2	3.63
7	fashion_roupa_masculina	3.64
8	telefonia_fixa	3.68
9	artigos_de_festas	3.77
10	fashion_roupa_feminina	3.78

Question 29 : Revenue vs Customer satisfaction ?

Query | Query History

```sql
SELECT p.product_category_name, ROUND(SUM(oi.price), 2) as category_revenue, ROUND(AVG(r.review_score), 2)
as avg_category_review
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
INNER JOIN reviews r ON oi.order_id = r.order_id
GROUP BY p.product_category_name
ORDER BY category_revenue DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 74

	product_category_name text	category_revenue numeric	avg_category_review numeric
1	beleza_saude	1252404.85	4.14
2	relogios_presentes	1197565.48	4.02
3	cama_mesa_banho	1040140.31	3.90
4	esporte_lazer	986848.92	4.11
5	informatica_acessorios	914579.39	3.93
6	moveis_decoracao	729864.42	3.90
7	utilidades_domesticas	630058.22	4.06
8	cool_stuff	629561.68	4.15
9	automotivo	586669.90	4.07
10	ferramentas_jardim	482946.89	4.04
11	brinquedos	479702.10	4.16
12	bebes	408716.75	4.01
13	perfumaria	398578.07	4.16
14	telefonia	320111.07	3.95

- We have some **High Revenue + High Reviews** : means Star Categories.
- We also have **High Revenue + Low Reviews** : means Danger Zone, Big money today, Risk tomorrow.
- And finally **Low Revenue + High Review**: means Growth Opportunity, Customers love them, Need more marketing.

7. Sellers analytics :

Question 30 : Total sellers ?

Query | Query History

```
1  SELECT COUNT(*) FROM sellers;
```

Data Output | Messages | Notifications

	count bigint
1	3095

Question 31: Top 10 sellers by revenue ?

Query | Query History

```
1  SELECT seller_id, ROUND(SUM(price),2) as revenue_by_seller
2  FROM order_items
3  GROUP BY seller_id
4  ORDER BY revenue_by_seller DESC
5  LIMIT 10;
```

Data Output | Messages | Notifications

	seller_id text	revenue_by_seller numeric
1	4869f7a5dfa277a7dca6462dcf3b52...	229472.63
2	53243585a1d6dc2643021fd1853d8...	222776.05
3	4a3ca9315b744ce9f8e93743614938...	200472.92
4	fa1c13f2614d7b5c4749cbc52fecda94	194042.03
5	7c67e1448b00f6e969d365cea6b010...	187923.89
6	7e93a43ef30c4f03f38b393420bc75...	176431.87
7	da8622b14eb17ae2831f4ac5b9dab...	160236.57
8	7a67c85e85bb2ce8582c35f2203ad7...	141745.53
9	1025f0e2d44d7041d6cf58b6550e0b...	138968.55
10	955fee9216a65b617aa5c0531780ce...	135171.70

- We can tell that Marketplace depends heavily on a small number of sellers.

Question 31 : Top 10 sellers by orders number ?

Query | Query History

```
1  SELECT seller_id, COUNT(DISTINCT order_id) as total_orders
2  FROM order_items
3  GROUP BY seller_id
4  ORDER BY total_orders DESC
5  LIMIT 10;
```

Data Output | Messages | Notifications

SQL

	seller_id text	total_orders bigint
1	6560211a19b47992c3666cc44a7e94...	1854
2	4a3ca9315b744ce9f8e93743614938...	1806
3	cc419e0650a3c5ba77189a1882b755...	1706
4	1f50f920176fa81dab994f9023523100	1404
5	da8622b14eb17ae2831f4ac5b9dab8...	1314
6	955fee9216a65b617aa5c0531780ce...	1287
7	7a67c85e85bb2ce8582c35f2203ad7...	1160
8	ea8482cd71df3c1969d7b9473ff13abc	1146
9	4869f7a5dfa277a7dca6462dcf3b52b2	1132
10	3d871de0142ce09b7081e2b9d1733c...	1080

Question 32: Average revenue per seller ?

```sql
SELECT ROUND(AVG(total_revenue),2) as average_revenue_per_seller
FROM
(SELECT seller_id, ROUND(SUM(price),2) AS total_revenue
FROM order_items
GROUP BY seller_id);
```

	average_revenue_per_seller numeric
1	4391.48

- We can use CTE for more cleaner version :

```sql
WITH revenue_cte as
(
SELECT seller_id, ROUND(SUM(price),2) as total_revenue
FROM order_items
GROUP BY seller_id
)

SELECT ROUND(AVG(total_revenue),2) as average_revenue_per_seller
FROM revenue_cte;
```

	average_revenue_per_seller numeric
1	4391.48

Question 33 : Seller revenue ranking ?

Query | Query History

```sql
WITH revenue_cte as
(
SELECT seller_id, ROUND(SUM(price),2) as total_revenue
FROM order_items
GROUP BY seller_id
)

SELECT seller_id,
    total_revenue,
    RANK() OVER (
        ORDER BY total_revenue DESC
    ) AS seller_rank
FROM revenue_cte
ORDER BY seller_rank;
```

Data Output | Messages | Notifications

SQL

	seller_id text	total_revenue numeric	seller_rank bigint
1	4869f7a5dfa277a7dca6462dcf3b52…	229472.63	1
2	53243585a1d6dc2643021fd1853d8…	222776.05	2
3	4a3ca9315b744ce9f8e93743614938…	200472.92	3
4	fa1c13f2614d7b5c4749cbc52fecda94	194042.03	4
5	7c67e1448b00f6e969d365cea6b010…	187923.89	5
6	7e93a43ef30c4f03f38b393420bc75…	176431.87	6
7	da8622b14eb17ae2831f4ac5b9dab8…	160236.57	7
8	7a67c85e85bb2ce8582c35f2203ad7…	141745.53	8
9	1025f0e2d44d7041d6cf58b6550e0b…	138968.55	9
10	955fee9216a65b617aa5c0531780ce…	135171.70	10

Question 34 : Top seller by state ?

Query | Query History

```sql
WITH seller_revenue AS
(
    SELECT s.seller_state, oi.seller_id, ROUND(SUM(oi.price),2) AS revenue
    FROM sellers s
    INNER JOIN order_items oi ON s.seller_id = oi.seller_id
    GROUP BY s.seller_state, oi.seller_id
),

ranked_sellers AS
(
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY seller_state
            ORDER BY revenue DESC
        ) AS rn
    FROM seller_revenue
)

SELECT * FROM ranked_sellers WHERE rn = 1;
```

Data Output | Messages | Notifications

SQL

	seller_state text	seller_id text	revenue numeric	rn bigint
1	AC	4be2e7f96b4fd749d52dff41f80e39...	267.00	1
2	AM	327b89b872c14d1c0be7235ef4871...	1177.00	1
3	BA	53243585a1d6dc2643021fd1853d8...	222776.05	1
4	CE	bbf9ad41dca6603e614efcdad7aab...	7846.00	1
5	DF	44073f8b7e41514de3b7815dd0237...	18729.64	1
6	ES	001cca7ae9ae17fb1caed9dfb1094...	25080.03	1
7	GO	0d5a9018aea56aeb367ba9e3f05d1...	10292.31	1

Question 35: Top sellers revenue share ?

Query Query History

```sql
WITH seller_revenue AS
(
    SELECT seller_id, ROUND(SUM(price),2) as revenue
    FROM order_items
    GROUP BY seller_id
    ORDER BY revenue DESC
    LIMIT 10
)

SELECT ROUND(SUM(revenue) * 100 / (SELECT SUM(price) FROM order_items),2) As top_sellers_revenue_share
FROM seller_revenue;
```

Data Output Messages Notifications

SQL Showing rows: 1 to 1

	top_sellers_revenue_share numeric
1	13.15

- Good thig we see that our business in not over concentrating , so that's good sign for not having marketplace concentration risk.

Question 36: Seller review performance ?

Query | Query History

```sql
SELECT oi.seller_id, ROUND(AVG(r.review_score),2) as avg_review
FROM order_items oi
INNER JOIN reviews r ON oi.order_id = r.order_id
GROUP BY oi.seller_id
ORDER BY avg_review DESC;
```

Data Output | Messages | Notifications

	seller_id text	avg_review numeric
1	3b18f9856c6eb2413eafedb58e9ee...	5.00
2	2ef086a599b597572aca4433b7ed6...	5.00
3	0b46f784306be7200ca1700aa55d8...	5.00
4	447d377bdb757058acb569025ee1...	5.00
5	fd435faa3c0422b60440ea3480d0e...	5.00
6	76833248bbc0e65a0293ec62023e...	5.00
7	a663d9c3797e90eac99ff60939416...	5.00
8	751e274377499a8503fd6243ad9c5...	5.00
9	a70e9066dbfd10b6b97a3f54a1356...	5.00
10	6ebf4eceee4dd9847201c82e77ef8...	5.00
11	a3b42d266fa8afc874b909422ce88...	5.00

Question 37 : Best sellers with high reviews?

Query Query History

```sql
WITH seller_metrics AS
(
    SELECT oi.seller_id,
        ROUND(SUM(oi.price),2) AS revenue,
        ROUND(AVG(r.review_score),2) AS avg_review
    FROM order_items oi
    INNER JOIN reviews r ON oi.order_id = r.order_id
    GROUP BY oi.seller_id
)

SELECT *
FROM seller_metrics
WHERE revenue > 50000
ORDER BY avg_review DESC
LIMIT 10;
```

Data Output Messages Notifications

	seller_id text	revenue numeric	avg_review numeric
1	e882b2a25a10b9c057cc49695f222...	52774.27	4.60
2	edb1ef5e36e0c8cd84eb3c9b003e4...	79284.55	4.43
3	fe2032dab1a61af8794248c819656...	65205.61	4.38
4	fa1c13f2614d7b5c4749cbc52fecda...	192774.43	4.34
5	37be5a7c751166fbc5f8ccba4119e0...	55525.54	4.30
6	ccc4bbb5f32a6ab2b7066a4130f114...	73654.72	4.28
7	f5a590cf36251cf1162ea35bef76fe84	53201.78	4.24
8	8581055ce74af1daba164fdbd55a40...	66054.50	4.23
9	7a67c85e85bb2ce8582c35f2203ad...	141130.58	4.23
10	f8db351d8c4c4c22c6835c19a46f01...	50386.80	4.22

- This combines high revenue with good customer satisfaction.

Question 38: High Revenue but Low Reviews ?

- We are going to use the same query buy reverse the reviews order:

Query | Query History

```sql
WITH seller_metrics AS
(
    SELECT oi.seller_id,
        ROUND(SUM(oi.price),2) AS revenue,
        ROUND(AVG(r.review_score),2) AS avg_review
    FROM order_items oi
    INNER JOIN reviews r ON oi.order_id = r.order_id
    GROUP BY oi.seller_id
)

SELECT *
FROM seller_metrics
WHERE revenue > 50000
ORDER BY avg_review ASC
LIMIT 10;
```

Data Output | Messages | Notifications

	seller_id text	revenue numeric	avg_review numeric
1	7c67e1448b00f6e969d365cea6b010...	188017.85	3.35
2	25c5c91f63607446a97b143d2d535d...	55267.08	3.70
3	4a3ca9315b744ce9f8e93743614938...	200561.42	3.80
4	966cb4760537b1404caedd472cc610...	51087.04	3.85
5	cca3071e3e9bb7d12640c9fbe23013...	64379.37	3.85
6	1025f0e2d44d7041d6cf58b6550e0bfa	139484.38	3.85
7	04308b1ee57b6625f47df1d56f00eedf	60130.60	3.88
8	522620dcb18a6b31cd7bdf73665113...	56096.70	3.90
9	7ddcbb64b5bc1ef36ca8c151f6ec77df	55266.59	3.90
10	6560211a19b47992c3666cc44a7e94...	122484.82	3.91

- These sellers are dangerous, they make money now, but may hurt: Customer retention, Brand reputation, Future sales

Question 39 : Seller delivery performance :

Query | Query History

```sql
SELECT oi.seller_id,
        AVG(
            o.order_delivered_customer_date - o.order_purchase_timestamp
        )  AS avg_delivery_time
FROM order_items oi
INNER JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_delivered_customer_date IS NOT NULL
GROUP BY oi.seller_id
ORDER BY avg_delivery_time ASC;
```

Data Output | Messages | Notifications

SQL

	seller_id text	avg_delivery_time interval
1	139157dd4daa45c25b0807ffff348...	1 day 05:08:25
2	5e063e85d44b0f5c3e6ec3131103a...	1 day 06:55:12
3	6561d6bf844e464b4019442692b4...	1 day 10:25:15
4	702835e4b785b67a084280efca35...	1 day 19:17:49
5	674207551483fec113276b67b0d8...	1 day 20:52:08
6	2c00c85d30361cd2ced2969cffbbff...	1 day 20:58:39
7	f3511c85f59f8dec53d140501ee8e...	1 day 21:59:45
8	96f7c797de9ca20efbe14545bed63...	1 day 22:42:34
9	751e274377499a8503fd6243ad9c...	1 day 23:56:21
10	e00d85ce20ea50c1224c66ca5050...	2 days 01:11:06
11	37303482a42fb700d8d127e70a9c...	2 days 01:14:28
12	7caa63f175b1cecbfaadd8b5ab999...	2 days 03:42:47
13	d9b00a97818674c7f8d8b1ef0e689...	2 days 04:52:32
14	7d81e74a4755b552267cd5e08156...	2 days 05:01:40

Question 40 : seller delivery performance vs seller reviews:

Query Query History

```sql
SELECT oi.seller_id,
        AVG(
            o.order_delivered_customer_date - o.order_purchase_timestamp
        )  AS avg_delivery_time,
        ROUND(
            AVG(r.review_score), 2
        )
FROM order_items oi
INNER JOIN orders o ON oi.order_id = o.order_id
INNER JOIN reviews r ON oi.order_id = r.order_id
WHERE o.order_delivered_customer_date IS NOT NULL
GROUP BY oi.seller_id
ORDER BY avg_delivery_time ASC
LIMIT 5;
```

Data Output Messages Notifications

SQL

	seller_id text	avg_delivery_time interval	round numeric
1	139157dd4daa45c25b0807ffff3483...	1 day 05:08:25	4.00
2	5e063e85d44b0f5c3e6ec3131103a...	1 day 06:55:12	5.00
3	6561d6bf844e464b4019442692b40...	1 day 10:25:15	5.00
4	702835e4b785b67a084280efca355...	1 day 19:17:49	5.00
5	674207551483fec113276b67b0d87...	1 day 20:52:08	5.00

- Here we can see sellers who deliver faster , have better average rating .

Question 41 : Sellers revenue vs delivery speed ?

Query | Query History

```sql
SELECT oi.seller_id,
        ROUND(
            SUM(oi.price), 2
        ) AS revenue,
        AVG(
            o.order_delivered_customer_date - o.order_purchase_timestamp
        )  AS avg_delivery_time

FROM order_items oi
INNER JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_delivered_customer_date IS NOT NULL
GROUP BY oi.seller_id;
```

Data Output | Messages | Notifications

	seller_id text	revenue numeric	avg_delivery_time interval
1	a61cc04793308395a84080710436...	14.63	8 days 08:51:02
2	6d2f2e3b539480db1e0842b3a4e32...	305.96	4 days 38:17:13.833333
3	c53bcd3be457a342a97e39e5a9f0b...	509.50	12 days 09:12:05.4
4	f9ec7093df3a7b346b7bcf7864069...	519.20	5 days 31:37:31.333333
5	fc38b5dceee1a730fad8853453437...	119.20	7 days 23:13:57.2
6	b39d7fe263ef469605dbb32608aee...	5063.20	12 days 31:12:34.375
7	5def4c3732941a971cba8fdee992e...	134.80	3 days 27:37:11.5
8	ea65d8b58316a6f2362f2a9e4b3e8...	2061.40	9 days 27:57:53.625
9	a45765f8afb1e594b22bf9e974b46...	79.99	4 days 23:26:28
10	be67f78487e2cecb0d55bc769709e...	102.00	10 days 00:49:52
11	71039d19d4303bf9054d69e9a923...	3537.20	12 days 33:19:44.7631...
12	3fd1e727ba94cfe122d165e176ce7...	2747.40	15 days 15:22:28.3333...

8. ADVANCED BUSINESS ANALYTICS :

In this phase we answers How is the business evolving over time?

8.1- Revenue Trend :

Question 42 : Monthly Revenue Trend ?

Query | Query History

```sql
SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
    ROUND(SUM(oi.price), 2) AS revenue
FROM orders o
INNER JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY month
ORDER BY month;
```

Data Output | Messages | Notifications

	month timestamp without time zone	revenue numeric
1	2016-09-01 00:00:00	267.36
2	2016-10-01 00:00:00	49507.66
3	2016-12-01 00:00:00	10.90
4	2017-01-01 00:00:00	120312.87
5	2017-02-01 00:00:00	247303.02
6	2017-03-01 00:00:00	374344.30
7	2017-04-01 00:00:00	359927.23
8	2017-05-01 00:00:00	506071.14
9	2017-06-01 00:00:00	433038.60
10	2017-07-01 00:00:00	498031.48
11	2017-08-01 00:00:00	573971.68
12	2017-09-01 00:00:00	624401.69
13	2017-10-01 00:00:00	664219.43
14	2017-11-01 00:00:00	1010271.37
15	2017-12-01 00:00:00	743914.17

Question 43: Orders monthly trend ?

Query | Query History

```sql
SELECT DATE_TRUNC('month', order_purchase_timestamp) AS month,
    COUNT(DISTINCT order_id) AS total_orders
FROM orders o
GROUP BY month
ORDER BY month;
```

Data Output | Messages | Notifications

	month timestamp without time zone	total_orders bigint
1	2016-09-01 00:00:00	4
2	2016-10-01 00:00:00	324
3	2016-12-01 00:00:00	1
4	2017-01-01 00:00:00	800
5	2017-02-01 00:00:00	1780
6	2017-03-01 00:00:00	2682
7	2017-04-01 00:00:00	2404
8	2017-05-01 00:00:00	3700
9	2017-06-01 00:00:00	3245
10	2017-07-01 00:00:00	4026
11	2017-08-01 00:00:00	4331
12	2017-09-01 00:00:00	4285
13	2017-10-01 00:00:00	4631
14	2017-11-01 00:00:00	7544
15	2017-12-01 00:00:00	5673
16	2018-01-01 00:00:00	7269

8.2- Running total :

Question 44: Cummulative Revenue ?

- How much revenue has the company generated over time ?

Query | Query History

```sql
WITH monthly_revenue AS
(
SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
    ROUND(SUM(oi.price), 2) AS revenue
FROM orders o
INNER JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY month
)

SELECT month,
    revenue,
    SUM(revenue) OVER (
            ORDER BY month
    ) AS cummulatice_revenue
FROM monthly_revenue;
```

Data Output | Messages | Notifications

SQL

	month timestamp without time zone	revenue numeric	cummulatice_revenue numeric
1	2016-09-01 00:00:00	267.36	267.36
2	2016-10-01 00:00:00	49507.66	49775.02
3	2016-12-01 00:00:00	10.90	49785.92
4	2017-01-01 00:00:00	120312.87	170098.79
5	2017-02-01 00:00:00	247303.02	417401.81
6	2017-03-01 00:00:00	374344.30	791746.11
7	2017-04-01 00:00:00	359927.23	1151673.34
8	2017-05-01 00:00:00	506071.14	1657744.48
9	2017-06-01 00:00:00	433038.60	2090783.08

8.3 – Moving average :

Question 45 : 3 months revenue moving average ?

Query Query History

```sql
WITH monthly_revenue AS
(
    SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
        ROUND(SUM(oi.price),2) as revenue
    FROM orders o
    INNER JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY month
)

SELECT month,
    revenue,
    ROUND(AVG(revenue) OVER (
            ORDER BY month
            ROWS BETWEEN 2 PRECEDING
            AND CURRENT ROW
    ),2) AS moving_average_3m
FROM monthly_revenue;
```

Data Output Messages Notifications

SQL

	month timestamp without time zone	revenue numeric	moving_average_3m numeric
1	2016-09-01 00:00:00	267.36	267.36
2	2016-10-01 00:00:00	49507.66	24887.51
3	2016-12-01 00:00:00	10.90	16595.31
4	2017-01-01 00:00:00	120312.87	56610.48
5	2017-02-01 00:00:00	247303.02	122542.26
6	2017-03-01 00:00:00	374344.30	247320.06
7	2017-04-01 00:00:00	359927.23	327191.52
8	2017-05-01 00:00:00	506071.14	413447.56

8.4 – Growth Analysis :

Question 46 : month-over-month growth ?

Query | Query History

```sql
WITH monthly_revenue AS
(
    SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
        ROUND(SUM(oi.price),2) as revenue
    FROM orders o
    INNER JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY month
)

SELECT month,
    revenue,
    LAG(revenue) OVER (
            ORDER BY month
    ) AS previous_month_revenue,
    ROUND(
            (revenue - LAG(revenue) OVER (ORDER BY month)) * 100
            /
            LAG(revenue) OVER (ORDER BY month),
            2
        ) AS growth_percentage
FROM monthly_revenue;
```

Data Output | Messages | Notifications

	month timestamp without time zone	revenue numeric	previous_month_revenue numeric	growth_percentage numeric
1	2016-09-01 00:00:00	267.36	[null]	[null]
2	2016-10-01 00:00:00	49507.66	267.36	18417.23
3	2016-12-01 00:00:00	10.90	49507.66	-99.98
4	2017-01-01 00:00:00	120312.87	10.90	1103687.80
5	2017-02-01 00:00:00	247303.02	120312.87	105.55

Question 47: Best growth month ?

Query | Query History

```sql
WITH growth AS
(
WITH monthly_revenue AS
(
    SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
        ROUND(SUM(oi.price),2) as revenue
    FROM orders o
    INNER JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY month
)

SELECT month,
    revenue,
    LAG(revenue) OVER (
            ORDER BY month
    ) AS previous_month_revenue,
    ROUND(
            (revenue - LAG(revenue) OVER (ORDER BY month)) * 100
            /
            LAG(revenue) OVER (ORDER BY month),
            2
        ) AS growth_percentage
FROM monthly_revenue
)

SELECT *
FROM growth
WHERE growth_percentage IS NOT NULL
ORDER BY growth_percentage DESC
LIMIT 1;
```

Data Output | Messages | Notifications

SQL

	month timestamp without time zone	revenue numeric	previous_month_revenue numeric	growth_percentage numeric
1	2017-01-01 00:00:00	120312.87	10.90	1103687.80

9. Data Mart (creating the views) :

1) View 1 – Monthly Revenue :

Query | Query History

```
1  CREATE OR REPLACE VIEW vw_monthly_revenue AS
2  SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
3      ROUND(SUM(oi.price),2) AS revenue,
4      COUNT(DISTINCT o.order_id) AS total_orders
5  FROM orders o
6  INNER JOIN order_items oi ON o.order_id = oi.order_id
7  GROUP BY month
8  ORDER BY month;
9
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 294 msec.
```

Query Query History

```
1  SELECT *
2  FROM vw_monthly_revenue;
3
4
5
```

Data Output Messages Notifications

	month timestamp without time zone	revenue numeric	total_orders bigint
1	2016-09-01 00:00:00	267.36	3
2	2016-10-01 00:00:00	49507.66	308
3	2016-12-01 00:00:00	10.90	1
4	2017-01-01 00:00:00	120312.87	789
5	2017-02-01 00:00:00	247303.02	1733
6	2017-03-01 00:00:00	374344.30	2641
7	2017-04-01 00:00:00	359927.23	2391
8	2017-05-01 00:00:00	506071.14	3660
9	2017-06-01 00:00:00	433038.60	3217
10	2017-07-01 00:00:00	498031.48	3969

2) View 2 – Customer Summary :

Query | Query History

```sql
CREATE OR REPLACE VIEW vw_customer_summary AS
SELECT c.customer_id,
    c.customer_state,
    COUNT(DISTINCT o.order_id) AS total_orders,
    ROUND(
        COALESCE(
            SUM(oi.price),
            0
        ),
        2
    ) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.customer_state;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 122 msec.
```

Query | Query History

```sql
SELECT *
FROM vw_customer_summary;
```

Data Output | Messages | Notifications

SQL

	customer_id text	customer_state text	total_orders bigint	total_spent numeric
1	00012a2ce6f8dcda20d059ce984917…	SP	1	89.80
2	000161a058600d5901f007fab4c271…	MG	1	54.90
3	0001fd6190edaaf884bcaf3d49edf079	ES	1	179.99
4	0002414f95344307404f0ace7a26f1…	MG	1	149.90
5	000379cdec625522490c315e70c7a…	SP	1	93.00
6	0004164d20a9e969af783496f34086…	SP	1	59.99
7	000419c5494106c306a97b5635748…	RJ	1	34.30
8	00046a560d407e99b969756e0b10f…	RJ	1	120.90

3) View 3 – Product Performance :

Query | Query History

```sql
CREATE OR REPLACE VIEW vw_product_performance AS
SELECT p.product_id,
    p.product_category_name,
    COUNT(oi.order_id) AS units_sold,
    ROUND(
        SUM(oi.price),
        2
    ) AS revenue,
    ROUND(
        AVG(oi.price),
        2
    ) AS average_price
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_category_name;

```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 205 msec.
```

Query | Query History

```sql
SELECT *
FROM vw_product_performance;

```

Data Output | Messages | Notifications

Showing rows

	product_id text	product_category_name text	units_sold bigint	revenue numeric	average_price numeric
1	00066f42aeeb9f3007548bb9d3f33c38	perfumaria	1	101.65	101.65
2	00088930e925c41fd95ebfe695fd2655	automotivo	1	129.90	129.90
3	0009406fd7479715e4bef61dd91f2462	cama_mesa_banho	1	229.00	229.00
4	000b8f95fcb9e0096488278317764d...	utilidades_domesticas	2	117.80	58.90
5	000d9be29b5207b54e86aa1b1ac548...	relogios_presentes	1	199.00	199.00
6	0011c512eb256aa0dbbb544d8dffcf6e	automotivo	1	52.00	52.00
7	00126f27c813603687e6ce486d909d...	cool_stuff	2	498.00	249.00
8	001795ec6f1b187d37335e1c470476...	consoles_games	9	350.10	38.90
9	001b237c0e9bb435f2e54071129237...	cama_mesa_banho	1	78.90	78.90
10	001b72dfd63e9833e8c02742adf472...	moveis_decoracao	14	489.86	34.99

4) View 4 – Seller performance :

Query | Query History

```sql
CREATE OR REPLACE VIEW vw_seller_performance AS
SELECT s.seller_id,
    s.seller_state,
    COUNT(DISTINCT oi.order_id) AS total_orders,
    ROUND(
        SUM(oi.price),
        2
    ) AS revenue,
    ROUND(
        AVG(r.review_score),
        2
    ) AS average_reviews
FROM sellers s
INNER JOIN order_items oi ON s.seller_id = oi.seller_id
LEFT JOIN reviews r ON oi.order_id = r.order_id
GROUP BY s.seller_id, s.seller_state;

```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 193 msec.
```

Query | Query History

```
1  SELECT *
2  FROM vw_seller_performance;
```

Data Output | Messages | Notifications

	seller_id text	seller_state text	total_orders bigint	revenue numeric	average_reviews numeric
1	0015a82c2db000af6aaaf3ae2ecb0...	SP	3	2685.00	3.67
2	001cca7ae9ae17fb1caed9dfb1094...	ES	200	25080.03	3.90
3	001e6ad469a905060d959994f1b41...	RJ	1	250.00	1.00
4	002100f778ceb8431b7a1020ff7ab4...	SP	51	1254.40	3.98
5	003554e2dce176b5555353e4f3555...	GO	1	120.00	5.00
6	004c9cd9d87a3c30c522c48c4fc07...	SP	158	20467.18	4.13
7	00720abe85ba0859807595bbf045a...	SP	13	1007.50	3.65
8	00ab3eff1b5192e5f1a63bcecfee11...	SP	1	98.00	5.00
9	00d8b143d12632bad99c0ad66ad5...	MG	1	86.00	5.00
10	00ee68308b45bc5e2660cd833c3f8...	SP	135	20345.00	4.33
11	00fc707aaaad2d31347cf883cd2dfe...	PR	103	13409.90	3.74
12	010543a62bd80aa422851e79a3bc7...	SP	2	1416.00	4.50
13	010da0602d7774602cd1b3f5fb7b7...	SP	1	849.50	1.00
14	011b0eaba87386a2ae96a7d32bb53...	SP	1	99.98	4.00
15	01266d4c46afa519678d16a8b683d...	PR	3	90.25	5.00
16	013900e863eace745d3ec7614cab5...	PR	23	1619.70	4.73
17	014c0679dd340a0e338872e7ec856...	MG	14	7058.00	3.67
18	014d9a685fd57276679edd00e0708...	PR	11	729.88	3.83
19	0176f73cc1195f367f7b32db1e5b3a...	SP	38	4733.10	4.03

5) View 5 – Delivery performance :

Query | Query History

```sql
CREATE OR REPLACE VIEW vw_delivery_performance AS
SELECT order_id,
    order_purchase_timestamp,
    order_delivered_customer_date,
    ROUND(
        EXTRACT(
            DAY FROM(
                order_delivered_customer_date - order_purchase_timestamp
            )
        ),
        2
    ) AS delivery_days
FROM orders
WHERE order_delivered_customer_date IS NOT NULL;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 155 msec.
```

Query | Query History

```sql
SELECT *
FROM vw_delivery_performance;
```

Data Output | Messages | Notifications

Showing rows:

	order_id text	order_purchase_timestamp timestamp without time zone	order_delivered_customer_date timestamp without time zone	delivery_days numeric
1	e481f51cbdc54678b7cc49136f2d6af7	2017-10-02 10:56:33	2017-10-10 21:25:13	8.00
2	53cdb2fc8bc7dce0b6741e21502734...	2018-07-24 20:41:37	2018-08-07 15:27:45	13.00
3	47770eb9100c2d0c44946d9cf07ec6...	2018-08-08 08:38:49	2018-08-17 18:06:29	9.00
4	949d5b44dbf5de918fe9c16f97b45f8a	2017-11-18 19:28:06	2017-12-02 00:28:42	13.00
5	ad21c59c0840e6cb83a9ceb5573f81...	2018-02-13 21:18:39	2018-02-16 18:17:02	2.00
6	a4591c265e18cb1dcee52889e2d8a...	2017-07-09 21:57:05	2017-07-26 10:57:55	16.00
7	6514b8ad8028c9f2cc2374ded24578...	2017-05-16 13:10:30	2017-05-26 12:55:51	9.00
8	76c6e866289321a7c93b82b54852d...	2017-01-23 18:29:09	2017-02-02 14:08:10	9.00
9	e69bfb5eb88e0ed6a785585b27e16d...	2017-07-29 11:55:02	2017-08-16 17:14:30	18.00
10	e6ce16cb79ec1d90b1da9085a6118...	2017-05-16 19:41:10	2017-05-29 11:18:31	12.00

6) View 6 – Category performance :

Query | Query History

```sql
CREATE OR REPLACE VIEW vw_category_performance AS
SELECT p.product_category_name,
    COUNT(DISTINCT oi.order_id) AS total_orders,
    ROUND(
        SUM(oi.price),
        2
    ) AS revenue,
    ROUND(
        AVG(r.review_score),
        2
    ) AS average_reviews
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
LEFT JOIN reviews r ON oi.order_id = r.order_id
GROUP BY p.product_category_name;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 167 msec.
```

Query | Query History

```sql
SELECT *
FROM vw_category_performance;
```

Data Output | Messages | Notifications

SQL

	product_category_name text	total_orders bigint	revenue numeric	average_reviews numeric
1	agro_industria_e_comercio	182	72530.47	4.00
2	alimentos	450	29393.41	4.22
3	alimentos_bebidas	227	15270.47	4.32
4	artes	202	24202.64	3.94
5	artes_e_artesanato	23	1814.01	4.13
6	artigos_de_festas	39	4485.18	3.77

Executive Summary

This project analyzed the Olist E-Commerce marketplace using PostgreSQL and Power BI to identify revenue drivers, customer behavior patterns, seller performance, and operational trends.

The analysis revealed that revenue is concentrated among a limited number of product categories and sellers, highlighting both growth opportunities and concentration risks. Customer spending patterns indicate the importance of retention strategies focused on high-value customers, while product and seller performance analyses uncovered opportunities to improve customer satisfaction and marketplace quality.

Operational analysis showed generally stable delivery performance, although certain regions experienced longer fulfillment times. Overall, the marketplace demonstrates strong growth potential supported by a diverse customer base and scalable operational processes.

The resulting Power BI dashboard provides decision-makers with an integrated view of business performance, enabling data-driven decisions across sales, marketing, operations, and customer experience functions.